

Git + GitHub: шпаргалка новичка

Команды и сценарии, которые мы уже прошли · обновлено 05.09.2026

Главная цепочка: ИЗМЕНИЛА -> ADD -> COMMIT -> PUSH

1. Навигация в Terminal

```
pwd
```

Показать, в какой папке я сейчас нахожусь.

```
ls
```

Показать файлы и папки в текущем месте.

```
ls -d */
```

Показать только папки в текущем месте.

```
cd Desktop
```

Перейти в папку Desktop.

```
cd ~/Desktop/github-learning
```

Сразу перейти в конкретную папку. Символ ~ означает домашнюю папку пользователя.

```
mkdir github-learning
```

Создать новую папку github-learning.

```
touch README.md
```

Создать пустой файл README.md.

```
echo $SHELL
```

Посмотреть, какой shell используется, например zsh.

2. Проверка и настройка Git

```
git --version
```

Проверить, установлен ли Git, и узнать его версию.

```
git config --global user.name
```

Посмотреть имя автора, настроенное в Git.

```
git config --global user.name "Anna Lukonina"
```

Установить имя автора коммитов.

```
git config --global user.email
```

Посмотреть email, настроенный в Git.

```
git config --global user.email "email@example.com"
```

Установить email автора коммитов. --global означает: использовать настройку по умолчанию для Git-проектов на этом Mac.

3. Создание локального репозитория

```
git init
```

Превратить обычную папку в локальный Git-репозиторий. Git создаёт внутри скрытую служебную папку `.git`, где хранится история и настройки репозитория.

4. Проверка состояния

```
git status
```

Показать, что сейчас происходит в репозитории. Если непонятно, что делать дальше, это первая команда, которую стоит запускать.

Что может показать `git status`

Untracked files - файл существует, но Git его пока не отслеживает.

Changes not staged for commit - файл изменён, но изменения ещё не добавлены в staging area.

Changes to be committed - изменения уже подготовлены для следующего коммита.

nothing to commit, working tree clean - все текущие изменения сохранены в коммитах; рабочая папка чистая.

ahead of 'origin/main' by 1 commit - локально есть коммит, которого ещё нет на GitHub.

behind 'origin/main' by 1 commit - на GitHub есть коммит, которого ещё нет в локальной main.

5. Подготовка изменений

```
git add README.md
```

Добавить изменения README.md в staging area - подготовить их к следующему коммиту.

6. Создание коммита

```
git commit -m "Update README"
```

Сохранить подготовленные изменения в истории Git. Флаг `-m` означает message - сообщение коммита.

```
git commit -m "Add repository description"
```

Ещё один пример понятного сообщения коммита: коротко описывает, что изменилось.

7. Просмотр истории

```
git log --oneline
```

Показать историю коммитов в компактном формате: короткий ID + сообщение.

8. Работа с текстом файла через Terminal

```
echo "# GitHub Learning" > README.md
```

Записать текст в файл. Символ `>` перезаписывает содержимое файла.

```
echo "Practice repository for learning Git." >> README.md
```

Добавить строку в конец файла. Символ `>>` дописывает, а не перезаписывает.

9. Подключение GitHub

```
git remote add origin https://github.com/annalukonina/github-learning.git
```

Связать локальный репозиторий с репозиторием на GitHub. `origin` - стандартное имя основного удалённого репозитория.

```
git remote -v
```

Посмотреть настроенные удалённые репозитории и адреса для fetch/push.

10. Первый push и связь веток

```
git push -u origin main
```

Первый раз отправить локальную ветку main в origin. Флаг -u запоминает связь main <-> origin/main.

```
git branch -vv
```

Показать локальные ветки, последний коммит и связь с удалёнными ветками.

11. Синхронизация с GitHub

```
git fetch
```

Узнать, что изменилось на GitHub, но не менять локальные файлы.

```
git pull
```

Получить изменения с GitHub и применить их к локальной ветке.

```
git push
```

Отправить свои локальные коммиты на GitHub.

12. Две схемы, которые нужно помнить

```
изменить файл -> git add -> staging area -> git commit -> git push
```

Рабочий путь, когда изменения делаются локально на Mac и затем отправляются на GitHub.

```
git fetch -> узнать | git pull -> получить | git push -> отправить
```

Самая короткая памятка по синхронизации.

main - твоя локальная ветка на Mac.

origin - имя основного удалённого репозитория GitHub.

origin/main - локальное представление того, в каком состоянии удалённая ветка main была при последней синхронизации.

13. Типичный рабочий цикл

Шаг	Что делаю	Команда / действие
1	Перейти в проект	cd ~/Desktop/github-learning
2	Проверить состояние	git status
3	Изменить файлы	работа с кодом / README
4	Проверить изменения	git status
5	Подготовить нужные изменения	git add README.md
6	Сохранить версию	git commit -m "Описание изменения"
7	Отправить на GitHub	git push

14. Ветки: безопасная работа отдельно от main

Ветка - отдельная линия изменений. Она позволяет тестировать функцию или гипотезу, не меняя стабильную main до момента merge.

```
git branch
```

Показать локальные ветки. Звёздочка * отмечает текущую ветку.

```
git branch experiment-readme
```

Создать новую локальную ветку, но остаться в текущей.

```
git switch experiment-readme
```

Переключиться на существующую ветку.

```
git switch -c feature-readme
```

Создать новую ветку и сразу переключиться на неё. -c = create.

```
git branch -vv
```

Показать ветки, последний коммит и связь с удалёнными ветками.

[origin/main: behind 2] - локальная main отстаёт от origin/main на 2 коммита.

[origin/feature-readme: gone] - удалённая ветка уже удалена, а локальная ещё существует.

```
git log --oneline --all --decorate --graph
```

Показать историю всех веток в виде компактного графика.

Merge и fast-forward

```
git merge experiment-readme
```

Влить указанную ветку в текущую ветку. Если текущая ветка не имеет своей отдельной линии коммитов, возможен fast-forward.

Fast-forward - Git не создаёт отдельный merge-коммит, а просто передвигает указатель текущей ветки вперёд по уже существующей прямой истории.

```
git branch -d experiment-readme
```

Удалить завершённую локальную ветку после merge. Коммиты не исчезнут, если они уже находятся в main.

15. Ветка на GitHub и Pull Request

```
git push -u origin feature-readme
```

Первый раз опубликовать feature-ветку на GitHub и связать её с origin/feature-readme.

branch -> изменения -> add -> commit -> push -> Pull Request -> review -> merge -> удалить branch

Pull Request (PR) - запрос на включение изменений из одной ветки в другую. В нашем случае: base = main, compare = feature-readme.

Conversation - обсуждение PR. Commits - коммиты PR. Checks - автоматические проверки. Files changed - diff, то есть конкретные изменения между ветками.

Diff - что именно изменилось

Diff показывает различия между двумя состояниями проекта: что добавлено, удалено или изменено. В PR зелёный + означает добавление, красный - означает удаление.

```
git diff
```

Показать локальные изменения, которые ещё не добавлены в staging area.

```
git diff --staged
```

Показать изменения, которые уже находятся в staging area и войдут в следующий коммит.

16. После merge PR: синхронизация и уборка веток

После merge на GitHub удалённая main может уйти вперёд, а локальная main ещё останется на старом коммите. Сначала обновляем сведения, затем локальную ветку.

```
git fetch --prune
```

Обновить сведения с GitHub и убрать локальные ссылки на удалённые ветки, которых на GitHub больше нет.

```
git switch main
```

Переключиться обратно в основную локальную ветку.

```
git pull
```

Подтянуть merge-коммит и другие новые изменения из origin/main в локальную main.

```
git branch -d feature-readme
```

Удалить завершённую локальную feature-ветку после того, как изменения уже попали в main.

17. Merge conflict: когда Git не может выбрать сам

Конфликт возникает, когда разные изменения затрагивают один и тот же участок файла, и Git не может автоматически решить, какой вариант оставить.

```
<<<<<< HEAD
локальная версия
=====
версия из origin/main
>>>>>> origin/main
```

Git помечает два конкурирующих варианта. Нужно вручную оставить правильный итоговый текст и удалить служебные маркеры.

исправить файл вручную -> git add -> git commit -> git push

```
git add README.md
```

После ручного исправления пометить конфликт в этом файле как разрешённый.

```
git commit -m "Resolve README merge conflict"
```

Зафиксировать результат разрешения конфликта в истории Git.

```
git push
```

Отправить итоговую историю на GitHub.

18. Clone: получить существующий репозиторий впервые

```
git clone https://github.com/annalukonina/github-learning.git
```

Создать новую локальную копию уже существующего GitHub-репозитория.

git clone скачивает файлы и историю коммитов, создаёт локальную main, автоматически настраивает origin и связь с origin/main.

После clone НЕ нужно выполнять git init и git remote add origin ...

git init = новый локальный репозиторий | git clone = локальная копия существующего удалённого репозитория

19. Полезные команды Terminal, которые встретились по ходу

```
rm README.md
```

Удалить файл. Мы использовали `rm` для ошибочно созданного неотслеживаемого файла. Команда удаляет файл с диска - использовать осторожно.

20. Отмена незакоммиченного изменения

```
git restore README.md
```

Вернуть файл к состоянию последнего коммита и отбросить незакоммиченные изменения. Использовать осторожно: несохранённая работа будет потеряна.

21. Быстрые сценарии: что делать в типовой ситуации

Ситуация	Что означает	Что делать
Changes not staged	Файл изменён, но ещё не staged	<code>git add README.md</code>
Changes to be committed	Изменения уже в staging area	<code>git commit -m "..."</code>
ahead of origin/main	Локально есть коммит, которого нет на GitHub	<code>git push</code>
behind origin/main	На GitHub есть новые коммиты	<code>git pull</code>
Нужно только проверить GitHub	Обновить origin/main без изменения файлов	<code>git fetch</code>
Удалённая ветка gone	Ветка удалена на GitHub, локальная ещё есть	<code>git fetch --prune</code>
Новый проект с GitHub	Репозитория ещё нет на этом Mac	<code>git clone URL</code>

Командный workflow через Pull Request

```
git switch -c feature-x -> изменить -> git add -> git commit -> git push -u origin feature-x -> PR -> review diff -> merge -> delete branch -> git pull
```

Ментальная модель origin/main

`main` - твоя локальная ветка. `origin` - имя удалённого репозитория. `origin/main` - локальный указатель на последнее известное Git состояние удалённой `main`. Реальная `main` на GitHub может измениться раньше, чем твой Mac узнает об этом; `git fetch` обновляет это знание.

Если не понимаешь, что происходит

```
1) pwd 2) git status 3) git branch -vv 4) при необходимости git fetch
```

Не угадывай команду по памяти. Сначала определи: где ты находишься, на какой ветке, есть ли незакоммиченные изменения и кто впереди - `local main` или `origin/main`.